

神经计算与类脑智能原理



李海峰 教授

类脑智能与神经工程研究中心

计算学部

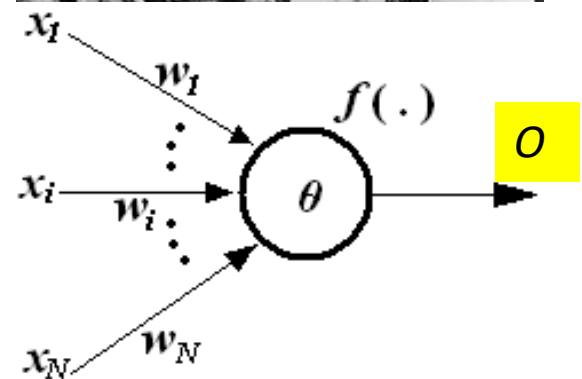
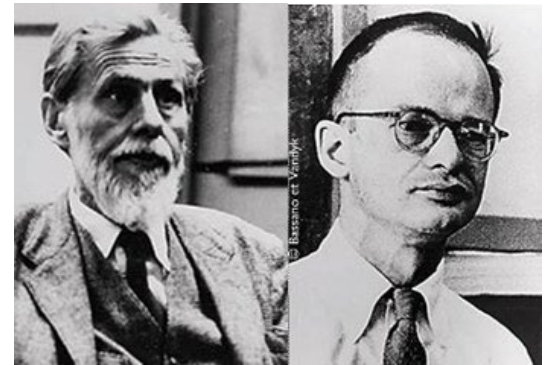


Email: lihaifeng@hit.edu.cn

CSA 1 孕育期：对人脑和智慧的早期模拟

1、最早的人工神经元 ---MP模型

- 1943年: McCulloch & Pitts提出
 - 精神生理学家 McCulloch
 - 数学家 Pitts
- MP模型本质上是神经网络逻辑演算部件
- 证明原则上可以计算任何可计算函数
- 标志着人工神经网络诞生



$$net(t) = \sum_{j=1}^N w_j \cdot x_j(t)$$

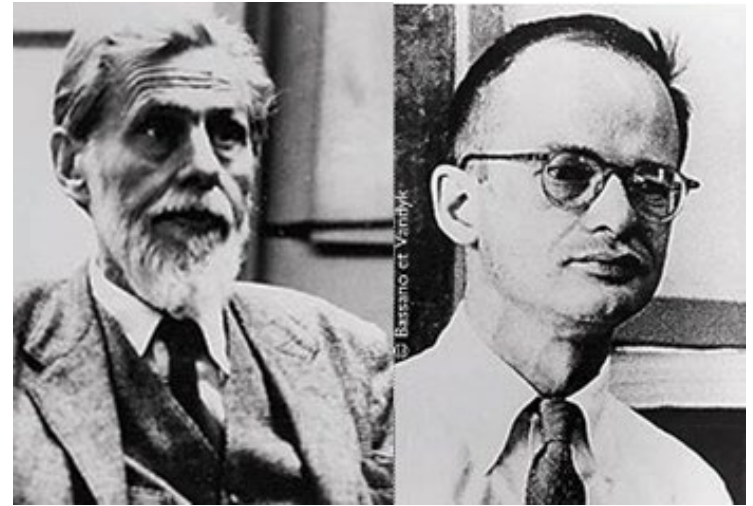
$$a(t) = F[a(t-1), net(t), \theta]$$

$$O(t) = f[a(t)]$$

CSA 1 孕育期：对人脑和智慧的早期模拟

1、最早的人工神经元 ---MP模型

- 1943年: McCulloch & Pitts提出
 - 精神生理学家 McCulloch
 - 数学家 Pitts

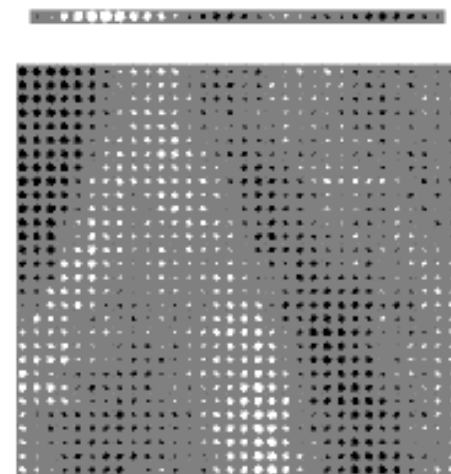
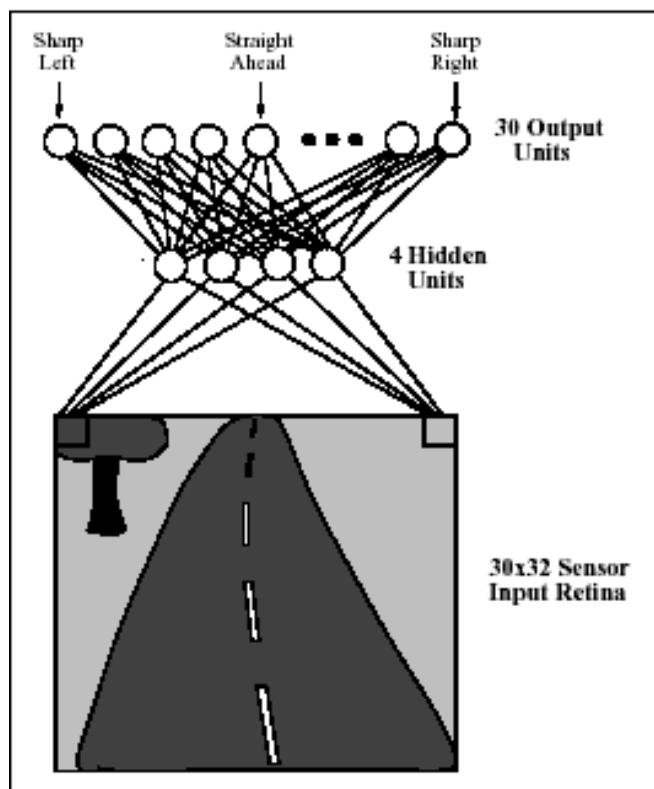


2、最早的机器学习方法 ---Hebb学习规则

1949年由D. Hebb提出，模拟大脑的学习，首次实现了人工神经网络的自学习。



- ALVINN *by* Pomerleau [1993]
 - drives 70 mph on highways for 95 miles



- 核心思想

- 采用“突触修正”学说

- 无教师学习 (Unsupervised learning)

- Hebb's rule

- 二值型: -1, +1

- 学习率: $\alpha > 0$

$$w_{ij}(t+1) = w_{ij}(t) + \alpha o_i(t) o_j(t)$$

- 有教师学习 (Supervised learning)

- Widrow's Delta rule

- 连续型输出

- 已知期望输出

$$w_{ij}(t+1) = w_{ij}(t) + \alpha (d_i - o_i(t)) o_j(t)$$



CSA 2.4 感知器学习算法

- 感知器学习是无教师学习？有教师学习？

- 1) 感知器学习算法

- 基本思想： 根据实际输出的正确与否调整权值

$$W = (w_1, w_2, \dots, w_N, \theta); \quad X = (x_1, x_2, \dots, x_N, -1)$$

- 权的调整： 如每次只对一个训练样本进行调整

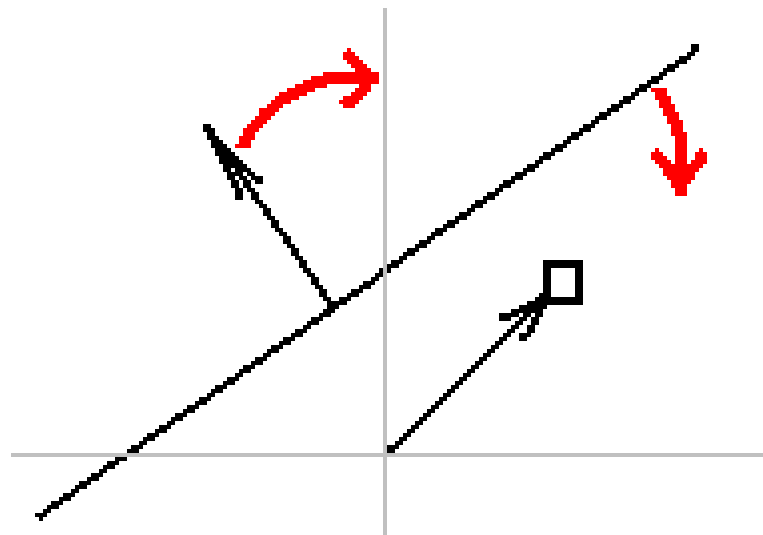
当输出不正确时， 则：

$$\text{if } : y = -1, \text{ then } : W = W + X$$

$$\text{if } : y = 1, \text{ then } : W = W - X$$



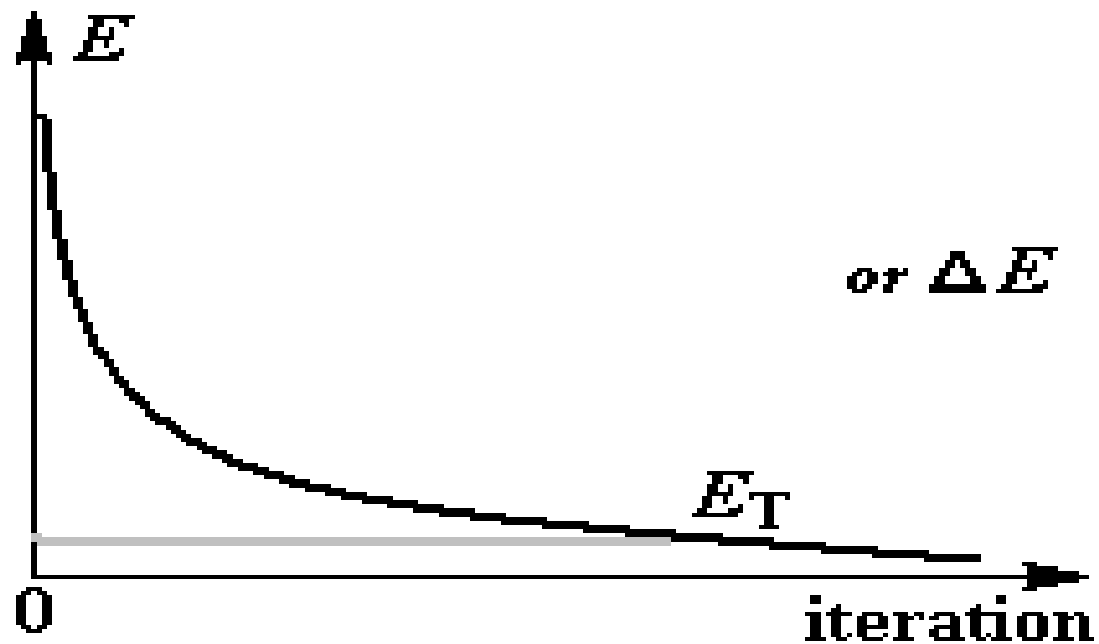
- 当 $y = -1$ 时:
 - 理想值应为 $d = 1$:
 - 相应的权应该得到加强
- 分界面向 X 方向旋转
 - Pattern Recognition理论



- (1) 初始化权值和阈值： 为小随机数
- (2) 开始一轮迭代
 - (3) 从训练集当中取一个样本
 - (4) 计算实际输出和输出误差
 - (5) 计算权和阈值的修正量
 - (6) 调整权和阈值
 - (7) 累计本轮总误差
- (8) 判断是否收敛？ 若否， 返回 (2)
- (9) 结束训练。



CSA 学习算法的终止条件

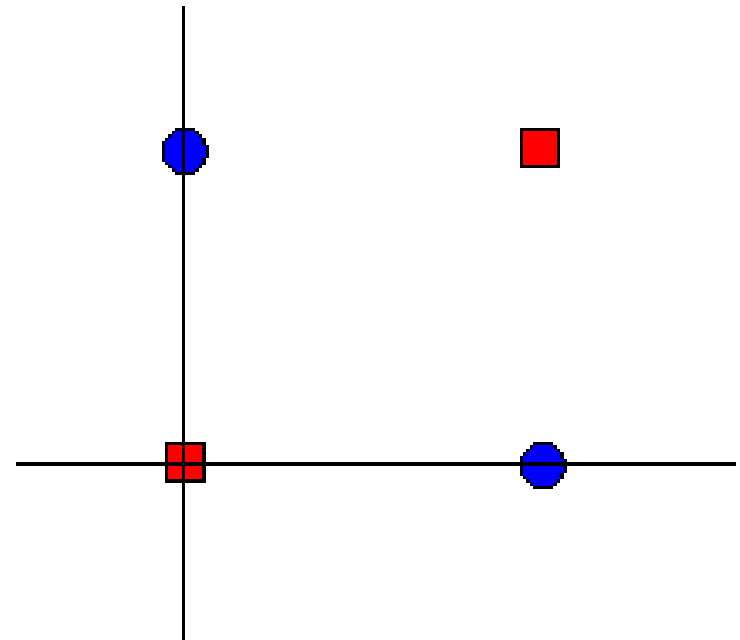
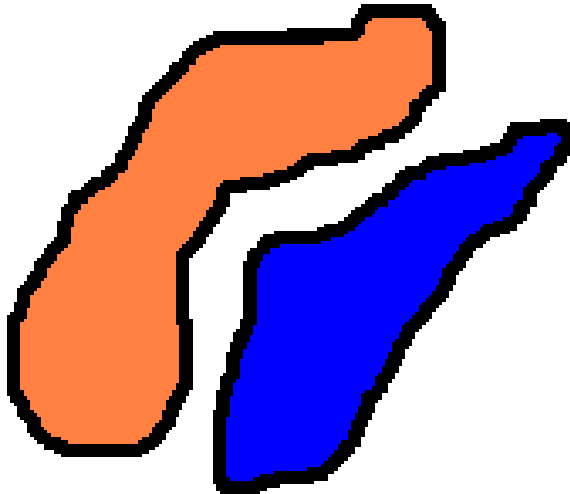


- 1) Max iterations
- 2) Global Error Threshold
- 3) Error Change Threshold



CSA Minsky's Problem

- Non-linear discriminative
- XOR vs. Minsky
 - M. Minsky and Papert, "Perceptron", 1969
 - Lack of "intra-representation"



- 再看线性不可分问题
 - 异或 (Exclusive - OR) 问题

$g(x, y)$		y	
		0	1
x	0	0	1
	1	1	0



线性可分与不可分函数全图

变量		函数及其值															
x	y	f ₁	f ₂	f ₃	f ₄	f ₅	f ₆	f ₇	f ₈	f ₉	f ₁₀	f ₁₁	f ₁₂	f ₁₃	f ₁₄	f ₁₅	f ₁₆
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1



CSA 一般问题线性不可分函数的数目

• R. O. Windner 于1960年研究

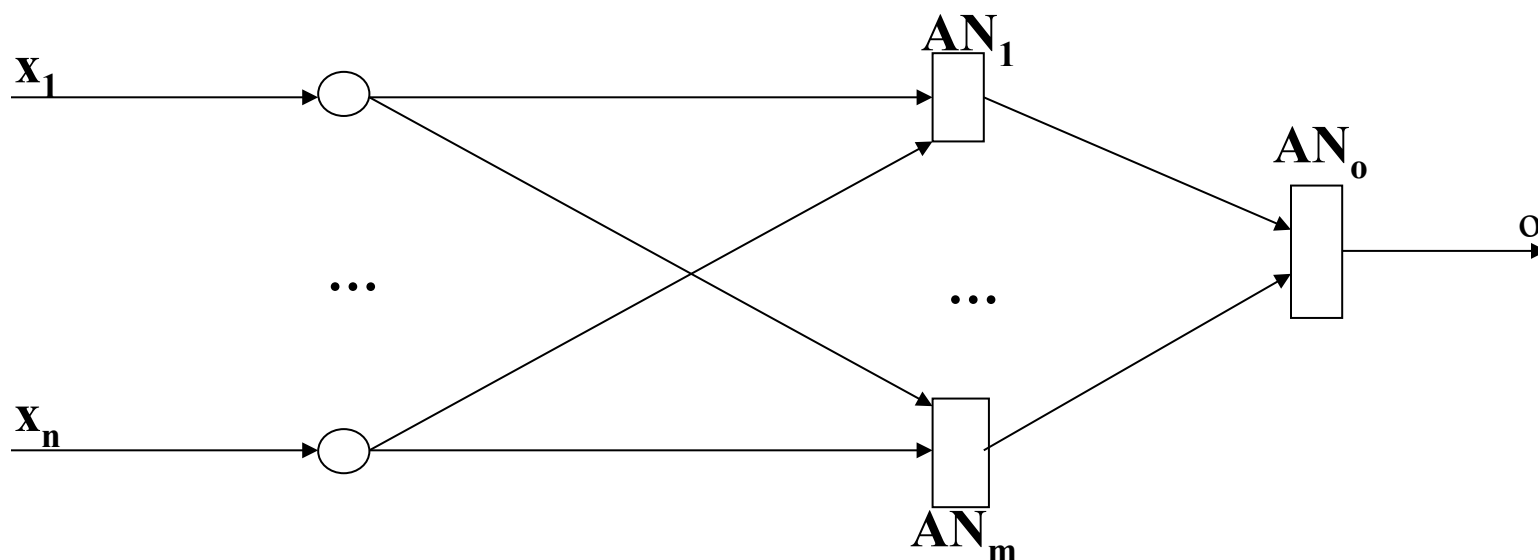
自变量个数	函数的个数	线性可分函数的个数
1	4 ?	4 ?
2	16	14 (87.5%)
3	256	104 (40.6%)
4	65,536	1882 (2.9%)
5	4.3×10^9	94,572 (0.002%)
6	1.8×10^{19}	5,028,134 (...)



- 用多个单级网组合在一起，并用其中的一个去综合其它单级网的结果，我们就可以构成一个两级网络，该网络可以被用来在平面上划分出一个封闭或者开放的凸域来
- 一个非凸域可以拆分成多个凸域。按照这一思路，三级网将会更一般一些，我们可以用它去识别出一些非凸域来。
- 解决好隐藏层的联接权的调整问题是非常关键的

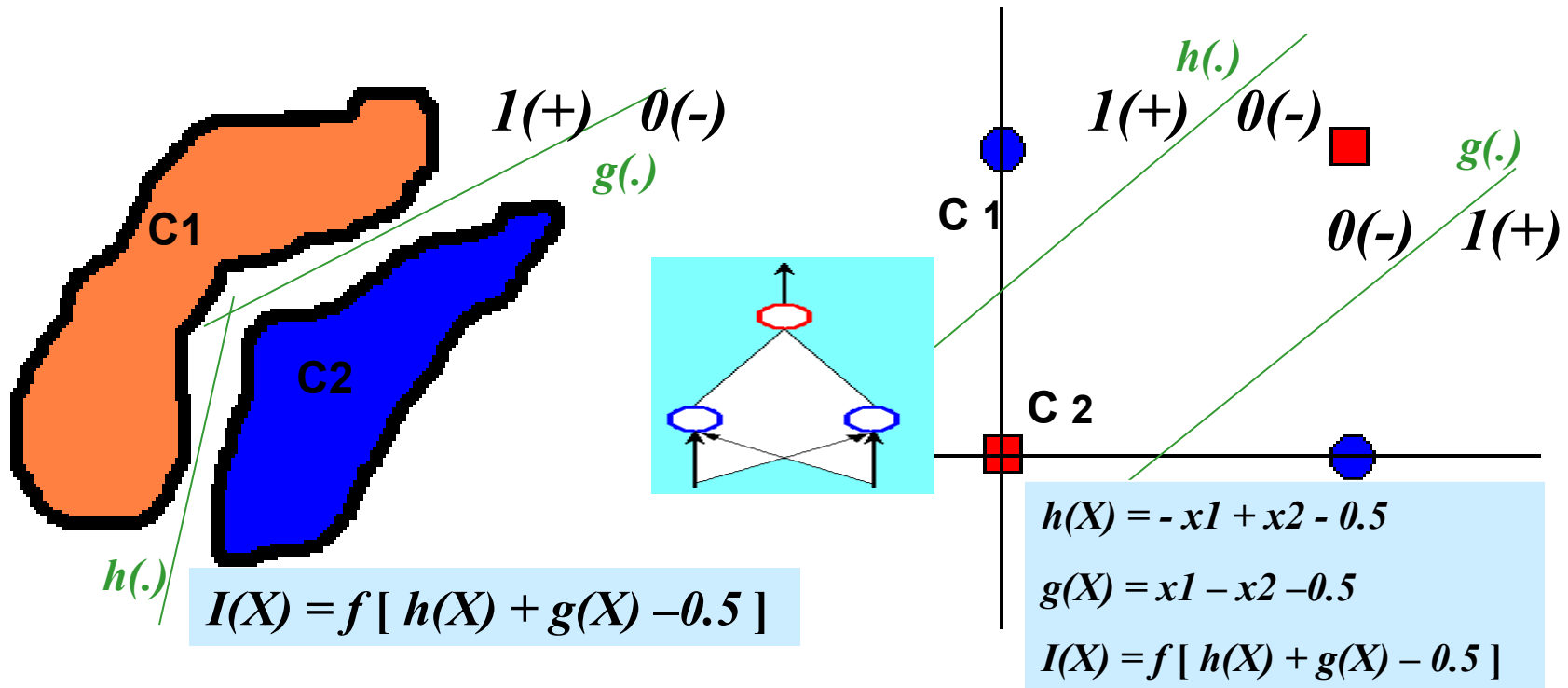


两级单输出网在n维空间中划分出m边凸域



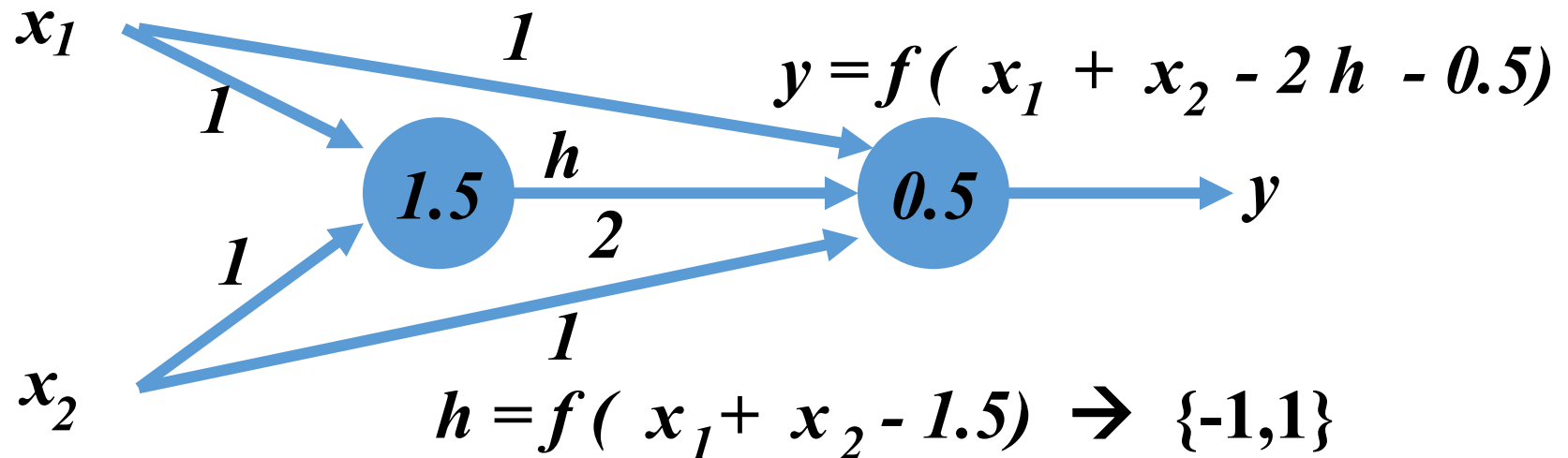
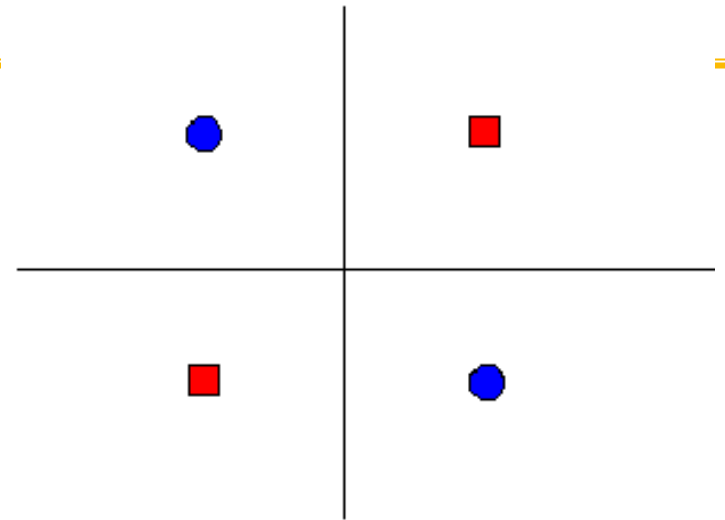
CSA Multi-Layer is a solution

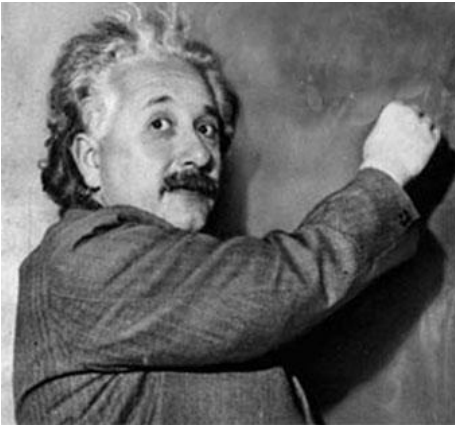
- Solution to non-linearly separable problem
 - Segmental linear decision surface
 - Multi-layer network



CSA XOR problem

- But how to obtain such a net?





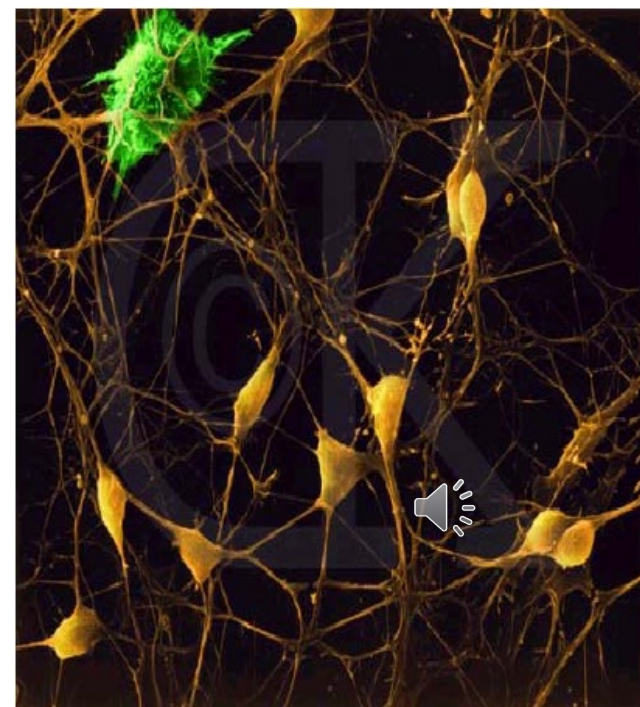
ANN-2: 多层感知器

MLP, BP, EBP, Feedforward, ...

人工神经元MP → 感知器P → 多层感知器MLP;

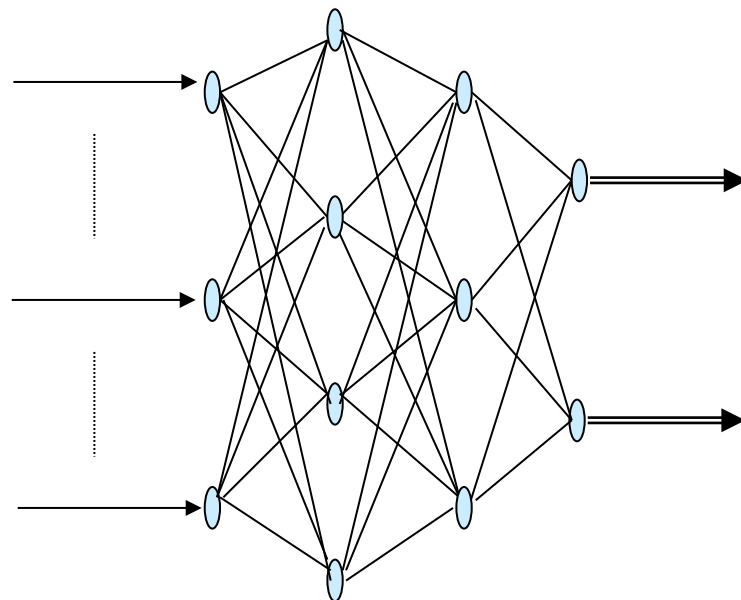
学习规则, EBP算法;

实用当中的问题; 等。



CSA (1) MLP Architecture

- Layered
 - No intra-layer connection
 - Full inter-layer connection
 - Feed forward only
- Neuron
 - M-P model $\rightarrow$ Sigmoid, Tanh...
- Notation
 - Input layer – Inputs
 - Hidden layer – 1st, 2nd ... hidden layer
 - Output layer – Outputs



• Feed forward

Given – an – example : $(X, D) = [(x_1, x_2, \dots, x_N), (d_1, d_2, \dots, d_R)]$

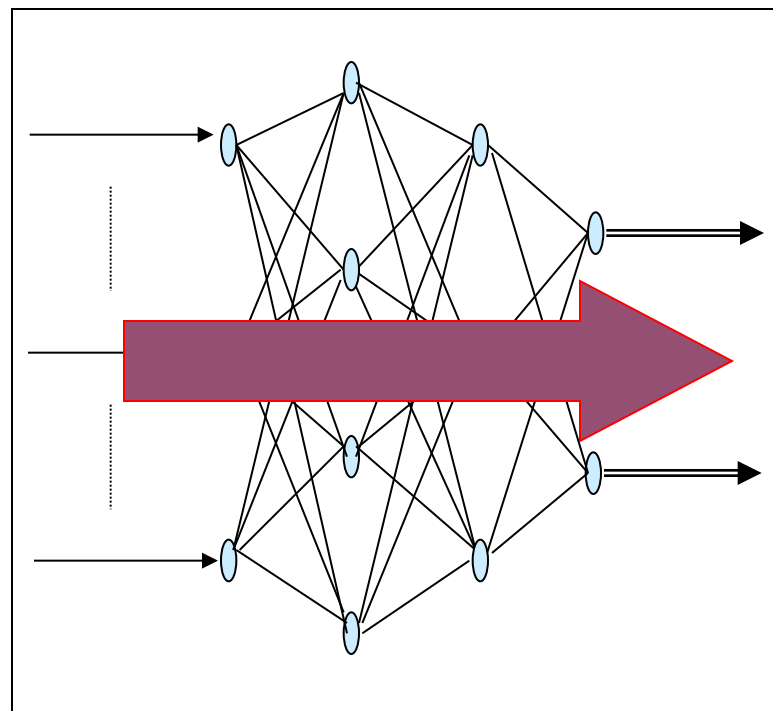
Inputs : $X = (x_1, x_2, \dots, x_N)$

Hidden – Layer :
$$h_i(X) = f^H \left(\sum_{j=1}^N w_{ij}^H x_j - \theta_i^H \right); i = 1, 2, \dots, K$$

Output – Layer :
$$o_i(X) = f^Y \left(\sum_{j=1}^K w_{ij}^Y h_j - \theta_i^Y \right); i = 1, 2, \dots, R$$

Outputs : $O = (o_1, o_2, \dots, o_R)$

Error :
$$e(X) = \|D - O\| = \frac{1}{2} \sum_{i=1}^R (d_i - o_i(X))^2$$

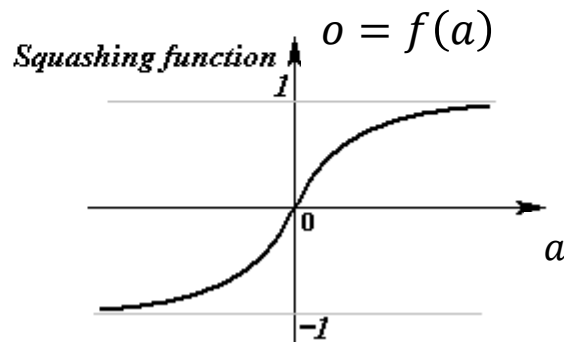
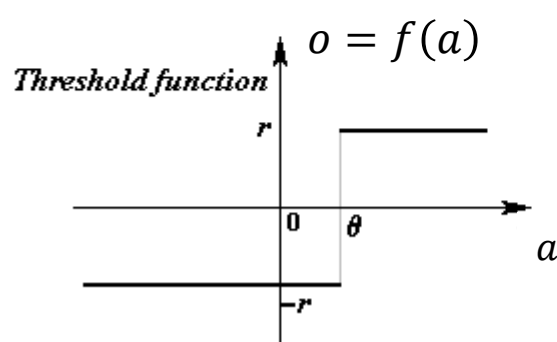
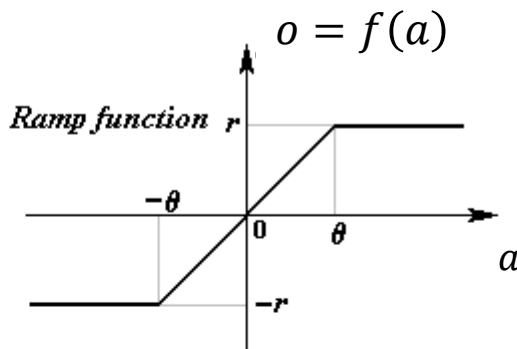
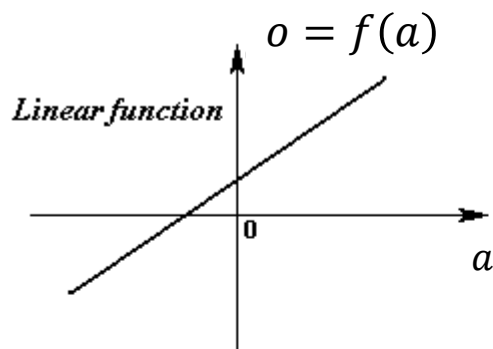




CSA 激活函数、激励函数、活化函数

- 一般要求

- 单调、增加、连续、可微函数
- 放大、限制、规正...



$\tanh$:

$$f(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$$

sigmoid :

$$f(a) = \frac{1}{1 + e^{-a}}$$

其他常见激活函数:

Relu、Gelu、Selu

Softmax ...



CSA Delta规则的另一种解释

• Widrow's δ 规则

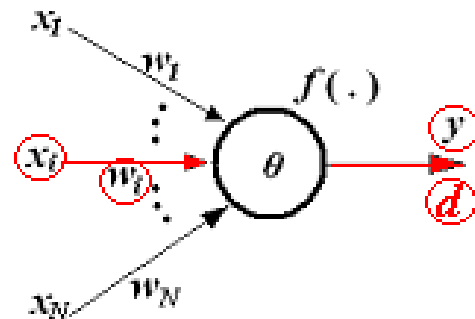
- 准则：使用实际输出 y_p 与理想输出 d_p 之间的误差来调整权

• 权的调整公式

- 如每次只对一个训练样本进行调整
- 则是调整后的输出误差减小？

$$w_i(t+1) = w_i(t) + \alpha(d_p - y_p)x_{pi}$$

$$\theta(t+1) = \theta(t) - \alpha(d_p - y_p)$$



• δ 规则的本质

• 准则:

- 使实际输出 y_p 与理想输 d_p 之间的误差能量最小

$$E = \sum_{p=1}^P E_p = \frac{1}{2} \sum_{p=1}^P (d_p - y_p)^2$$

■ 收敛性证明 – 梯度下降原理



CSA ANN就是一个非常好的工具

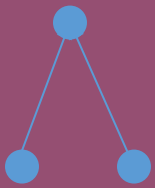
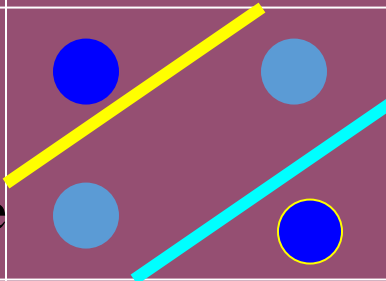
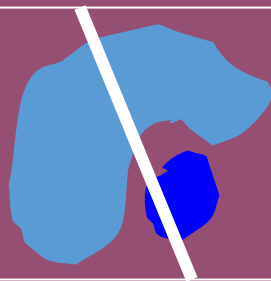
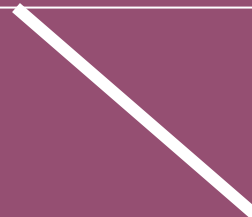
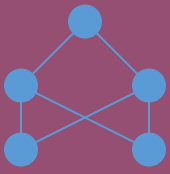
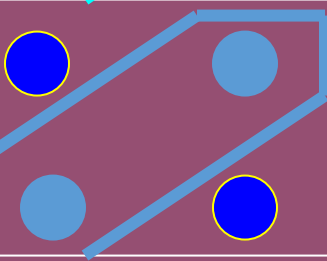
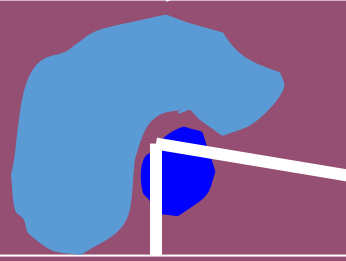
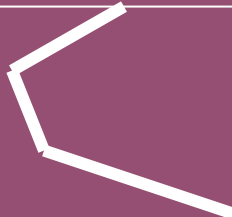
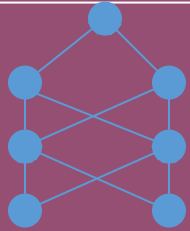
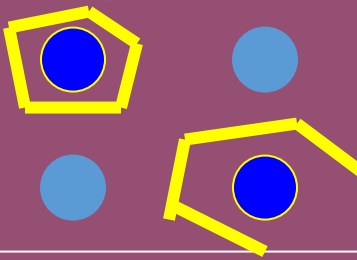
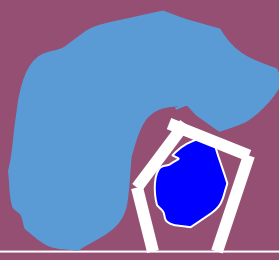
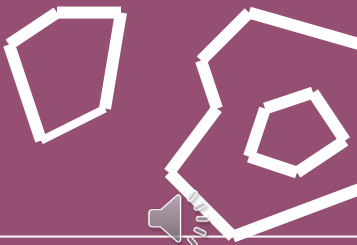
- ANN具有
 - 自学习能力
 - 强大的信息处理能力
 - 自组织（优化）结构能力
- 特别适用于
 - 智能信息处理与智能数据分析
 - 知识获取、预测、识别、分析、建模 ...



- **Linear**
 - Always linear combination
= single layer network
 - Reason: No jumps, no intra-connection
- **Squashing**
 - Sophisticate decision surfaces
- *Sigmoid / Tanh* is usually applied !!!
 - Or mixed types of activation function is used.



3.3 MLP Capacity & Performance

Network Architecture (Binary A.F.)	Types of the Classified Region	The Classification for XOR	Messed Classified Region	General form of Classified Region
	2 Regions Separated by a sphere-plane			
	Open Convex Region or Closed Convex Region			
	Arbitrary Forms			



CSA 3.4 Hidden Neuron Number

- MLP capacity conclusion
 - 3 layers is enough for any classifier.
 - 3-layer MLP may approach any continuous function in arbitrary accuracy.
- Neuron number for the worst case
 - 2nd hidden: number of the un-joined regions
 - 1st hidden:
 - at least 3 boundaries for each region
 - Pyramid structure
 - The later should be more than 3 times than the former.
- Sigmoid
 - Smooth, complex surfaces



- Single hidden layer MLP
 - $K \rightarrow \infty$, the network may approach any continuous mapping function

$$\forall \text{Conti.Func. } G: X \rightarrow Y, \quad \exists K, W^*: F_{W^*}(\cdot) \approx G(\cdot)$$

- How to choose a K ?
 - Experience
 - Trying
- Clipping and merging



- Supervised training

$$(X, D) = [(x_1, x_2, \dots, x_N), (d_1, d_2, \dots, d_R)]$$

- General delta rule

$$w_i = w_i + \Delta w_i$$

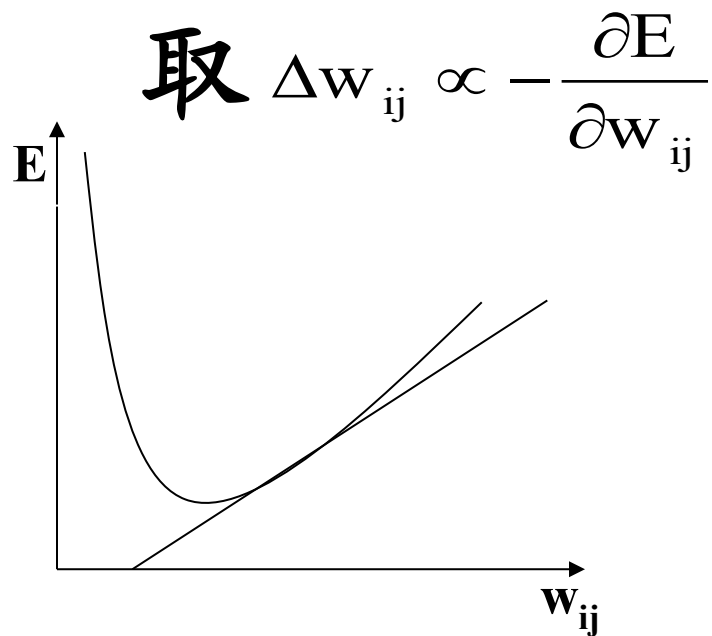
- Criteria

$$\text{Error:} \quad e(X) = \|D - O\| = \frac{1}{2} \sum_{i=1}^R (d_i - o_i(X))^2$$

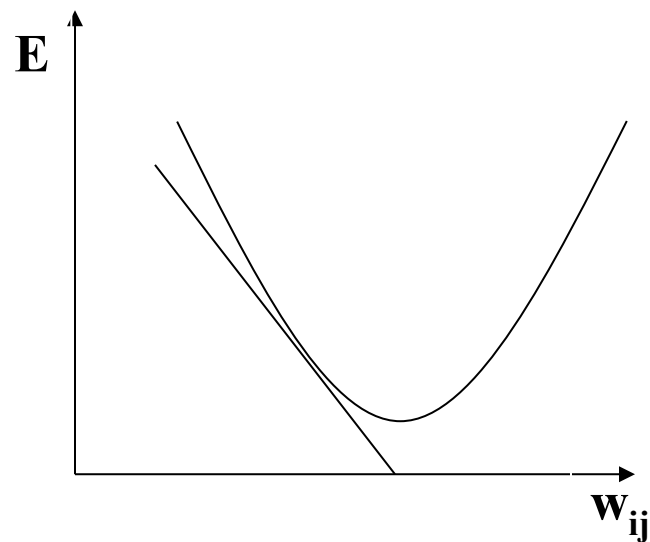
- Sigmoid activation function



最速下降法，要求E的极小点



$$\frac{\partial E}{\partial w_{ij}} > 0, \text{ 此时 } \Delta w_{ij} < 0$$



$$\frac{\partial E}{\partial w_{ij}} < 0, \text{ 此时 } \Delta w_{ij} > 0$$



CSA Newton's error minimizing method

$$\begin{aligned} \text{Error : } e(X) &= \|D - O\| = \frac{1}{2} \sum_{i=1}^R (d_i - o_i(X))^2 = \frac{1}{2} \sum_{i=1}^R (d_i - f^Y(\text{net}_i^Y(X)))^2 \\ &= \frac{1}{2} \sum_{i=1}^R \left(d_i - f^Y \left(\sum_{j=1}^K w_{ij}^Y h_j - \theta_i^Y \right) \right)^2 = F(w_{ij}, \theta_i) \end{aligned}$$

$$\text{To - min- } e(X) : w_{ij} \leftarrow w_{ij} + \Delta w_{ij}, \text{ where : } \Delta w_{ij} \propto -\frac{\partial e(X)}{\partial w_{ij}}$$

$\Rightarrow$

$$\Delta w_{ij} = -\alpha \frac{\partial e}{\partial w_{ij}} = -\alpha \frac{\partial e}{\partial o_i^l} \cdot \frac{\partial o_i^l}{\partial \text{net}_i} \cdot \frac{\partial \text{net}_i}{\partial w_{ij}} = -\alpha \frac{\partial e}{\partial o_i^l} \cdot \frac{\partial o_i^l}{\partial \text{net}_i} \cdot o_j^{l-1}$$

$$\Delta \theta_i = -\alpha \frac{\partial e}{\partial \theta_i} = -\alpha \frac{\partial e}{\partial o_i^l} \cdot \frac{\partial o_i^l}{\partial \text{net}_i} \cdot \frac{\partial \text{net}_i}{\partial \theta_i} = \alpha \frac{\partial e}{\partial o_i^l} \cdot \frac{\partial o_i^l}{\partial \text{net}_i}$$



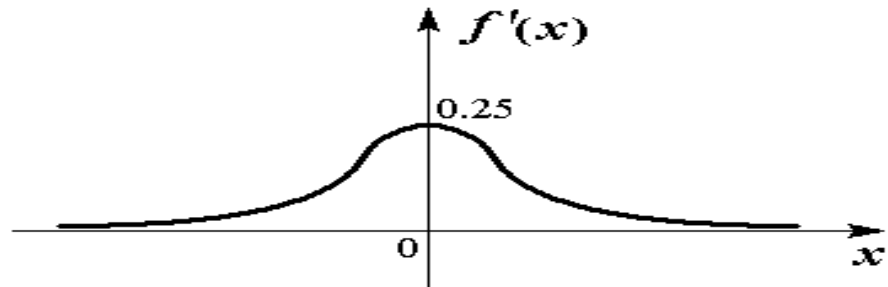
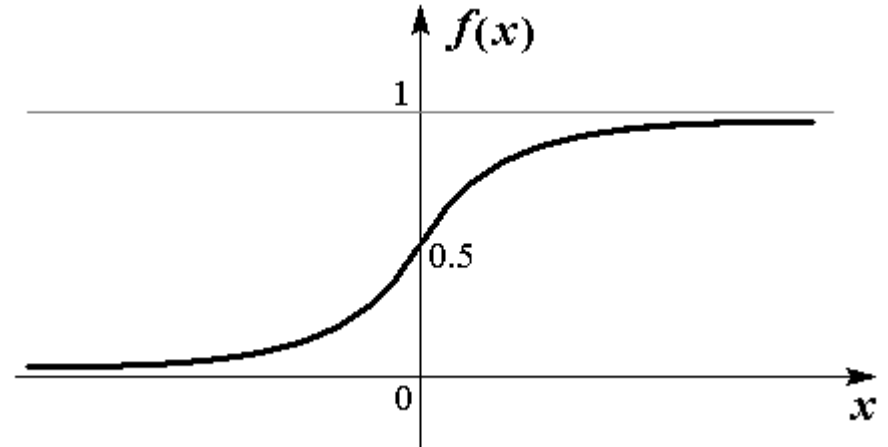
CSA Sigmoid activation function

- Derivative

- $f: (-\infty, +\infty) \rightarrow (0, 1)$

$$f(x) = \frac{1}{1 + e^{-x}} \quad f'(x) = f(x)[1 - f(x)]$$

$$\begin{aligned} f'(x) &= \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1 + e^{-x} - 1}{(1 + e^{-x})^2} \\ &= \frac{1}{1 + e^{-x}} - \frac{1}{(1 + e^{-x})^2} \\ &= f(x) - [f(x)]^2 \\ &= f(x)[1 - f(x)] \end{aligned}$$



CSA Δ weights for the output layer

$$\Delta w_{ij}^Y = -\alpha \frac{\partial e(X)}{\partial o_i(X)} \bullet \frac{\partial o_i(X)}{\partial net_i} \bullet h_j$$

$$\therefore -\frac{\partial e(X)}{\partial o_i(X)} = -\frac{\partial}{\partial o_i(X)} \left(\frac{1}{2} \sum_{i=1}^R (d_i - o_i(X))^2 \right) = d_i - o_i(X)$$

$$\frac{\partial o_i(X)}{\partial net_i} = f'(net_i) = f(net_i) [1 - f(net_i)] = o_i(X) [1 - o_i(X)]$$

$$\therefore \Delta w_{ij}^Y = \alpha \bullet o_i [1 - o_i] [d_i - o_i] h_j = \alpha \delta_i^Y h_j$$

$$\Delta \theta_i = -\alpha \bullet o_i [1 - o_i] [d_i - o_i] = -\alpha \delta_i^Y$$



CSA Δ weights for the hidden layer

$$\Delta w_{ij}^H = -\alpha \frac{\partial e}{\partial h_i} \bullet \frac{\partial h_i}{\partial \text{net}_i} \bullet x_j = -\alpha \frac{\partial e}{\partial h_i} \bullet h_i [1 - h_i] \bullet x_j$$

$$\therefore -\frac{\partial e}{\partial h_i} = -\sum_{r=1}^R \frac{\partial e}{\partial o_r} \bullet \frac{\partial o_r}{\partial h_i}$$

$$\frac{\partial o_r}{\partial h_i} = \frac{\partial}{\partial h_i} \left(f \left(\sum_{i=1}^K w_{ri}^Y h_i - \theta_r^Y \right) \right) = f' \left(\sum_{i=1}^K w_{ri}^Y h_i - \theta_r^Y \right) \bullet w_{ri}^Y = w_{ri}^Y o_r [1 - o_r]$$

$$-\frac{\partial e}{\partial o_r} = -\frac{\partial}{\partial o_r} \left(\frac{1}{2} \sum_{i=1}^R (d_i - o_i)^2 \right) = (d_r - o_r)$$

$$\Rightarrow -\frac{\partial e}{\partial h_i} = \sum_{r=1}^R w_{ri}^Y o_r [1 - o_r] (d_r - o_r) = \sum_{r=1}^R w_{ri}^Y \delta_r^Y$$

$$\therefore \Delta w_{ij}^Y = \alpha \bullet h_i [1 - h_i] \bullet \sum_{r=1}^R w_{ri}^Y \delta_r^Y \bullet x_j = \alpha \delta_i^H x_j; \quad \Delta \theta_i = -\alpha \delta_i^H$$

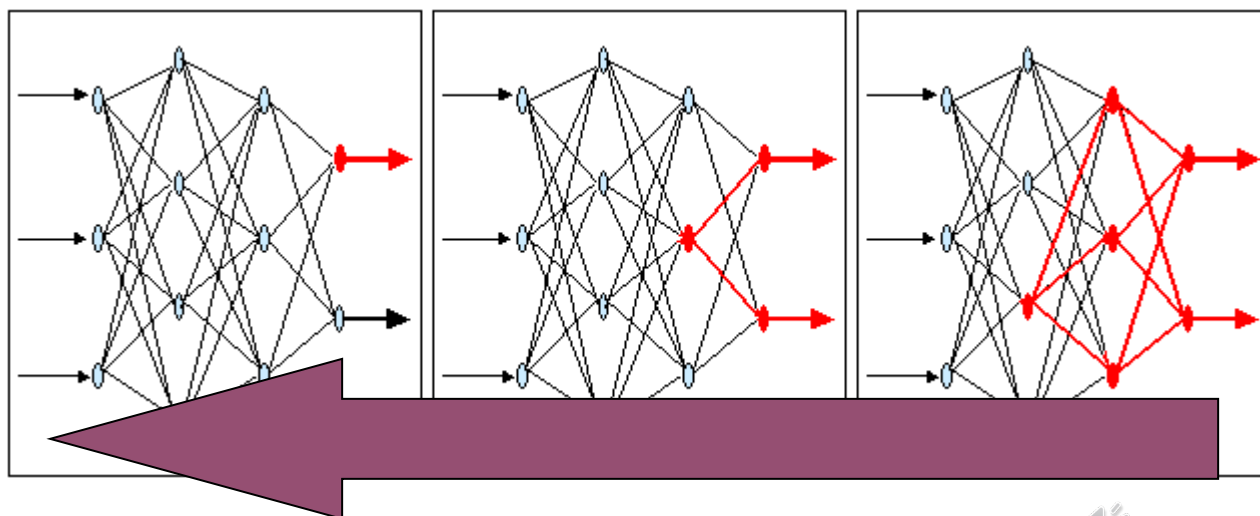


CSA Feedback of errors

- 2 operational stages
 - Working
 - Training

$$\delta_r^Y = o_r [1 - o_r] \bullet [d_r - o_r]$$

$$\delta_i^H = h_i [1 - h_i] \bullet \sum_{r=1}^R w_{ri}^Y \delta_r^Y$$



CSA EBP Algorithm

- (1) Initialize weights and thresholds to small random values: $(-0.1, 0.1)$.
- (2) Consider one training example:

$$(X_t, D_t) = ((x_1(t), x_2(t), \dots, x_N(t)), (d_1(t), d_2(t), \dots, d_R(t)))$$

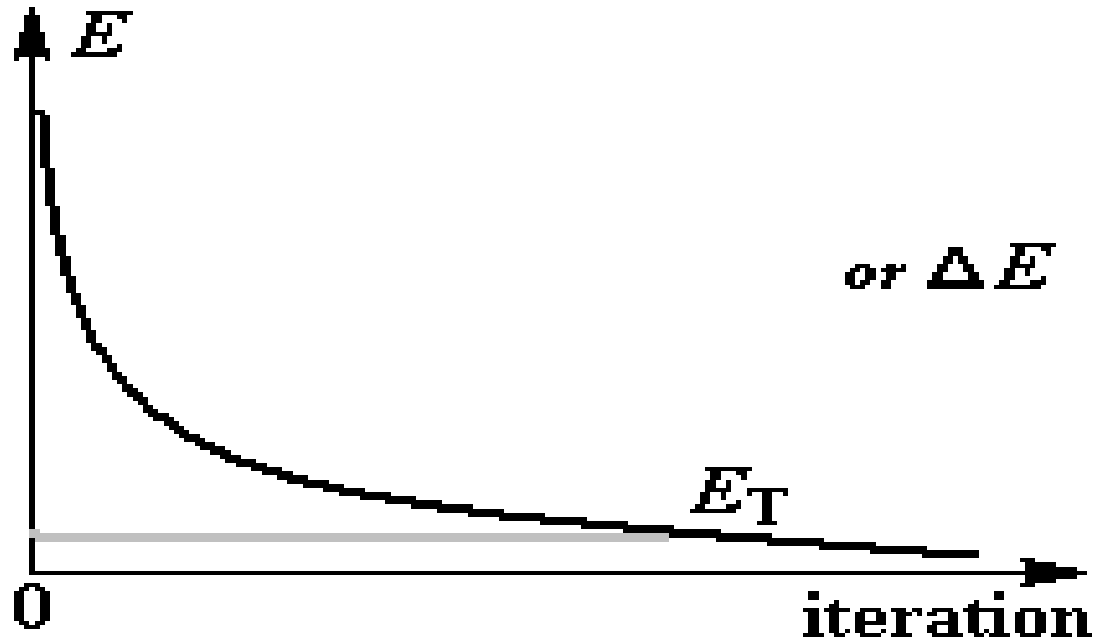
$$\text{where: } d_r(t) = \begin{cases} 0.9, & \text{if } : X_t \in C_r \\ 0.1, & \text{if } : X_t \notin C_r \end{cases}$$

- (3) Feed forward: calculate the output of each node.
- (4) Feed backward: calculate the error of each node.
- (5) Modify weights and thresholds.
- (6) Accumulate error: $E = E + e (W(t-1), X_t)$
- (7) Repeat (2-6), till convergence.

Possible non-monotone decline of E



CSA Termination---学习算法的终止条件



- 1) Max iterations
- 2) Global Error Threshold
- 3) Error Change Threshold



本小节结束，谢谢大家！

Thank You !

